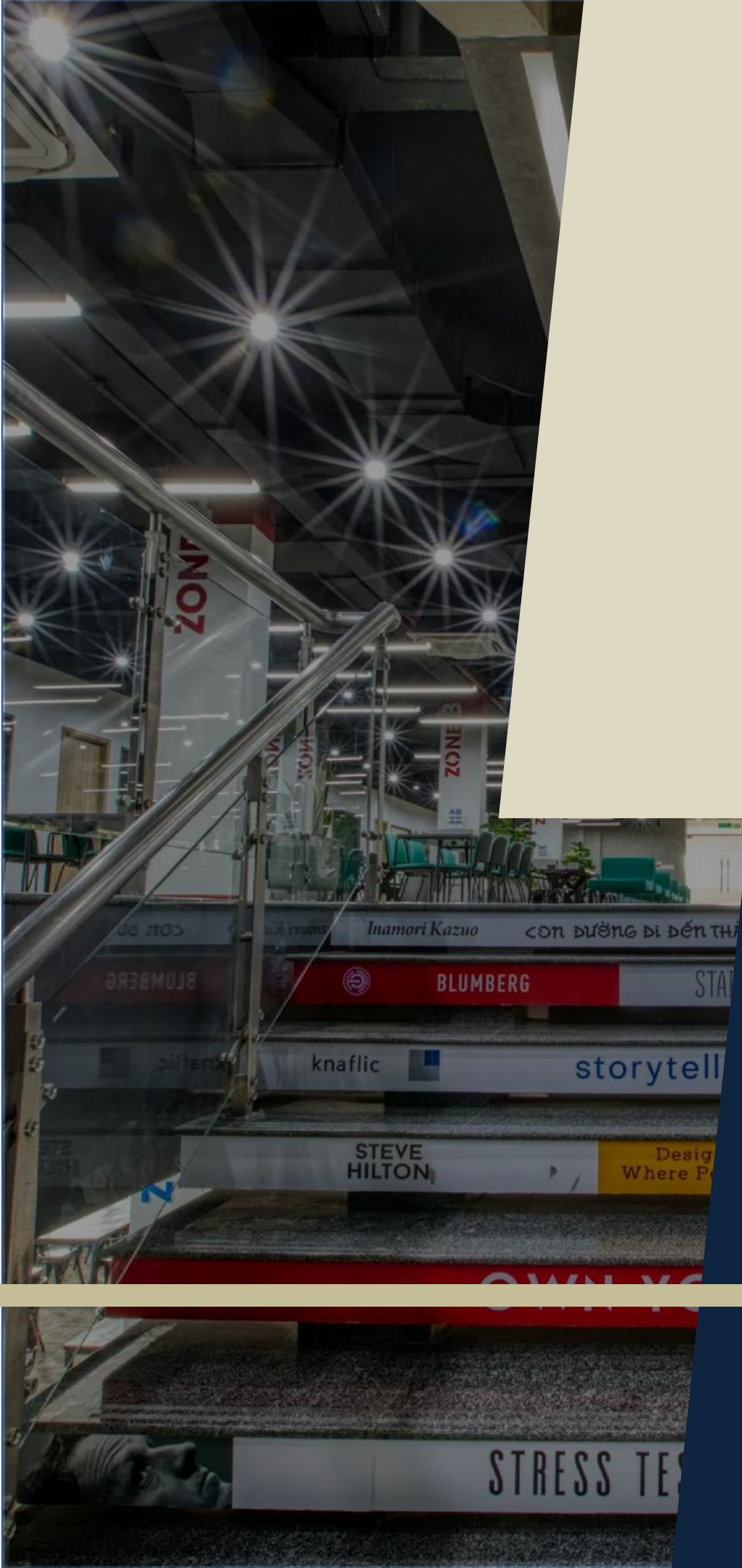


Coding Practice

SOLID Principles



SOLID Principles

Contents

- What is SOLID
- Single responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

What is SOLID?

S

SRP

Single
Responsibility
Principle

O

OCP

Open/Closed
Principle

L

LSP

Liskovs
Substitution
Principle

I

ISP

Interface
Segregation
Principle

D

DIP

Dependency
Inversion
Principle

Single Responsibility Principle

*“A class should have **one and only one** reason to change”*

Single Responsibility Principle

```
6 public class Employee
7 {
8     public double CalculatePay(Money money)
9     {
10         //business logic for payment here
11     }
12
13     public Employee Save(Employee employee)
14     {
15         //store employee here
16     }
17 }
```

Business logic



Persistence



There are **two** responsibilities



Single Responsibility Principle

How to solve this?



Single Responsibility Principle

```
21 public class Employee
22 {
23     public double CalculatePay(Money money)
24     {
25         //business logic for payment here
26     }
27 }
28
29 public class EmployeeRepository
30 {
31     public Employee Save(Employee employee)
32     {
33         //store employee here
34     }
35 }
```



Just create two different classes

Open/Closed Principle

*“Software entities should
be **open** for extension, but
closed for modification.”*

Open/Closed Principle

- Increased stability – existing code (almost) never changes
- Increased modularity, but many small classes

Open/Closed Principle

```
40 public enum PaymentType = { Cash, CreditCard };
41
42 public class PaymentManager
43 {
44     public PaymentType PaymentType { get; set; }
45
46     public void Pay(Money money)
47     {
48         if(PaymentType == PaymentType.Cash)
49         {
50             //some code here - pay with cash
51         }
52         else
53         {
54             //some code here - pay with credit card
55         }
56     }
57 }
```



Humm...and if I need to add a new payment type?

You need to modificate this class.

Open/Closed Principle



open for
extension

close for
modification

```
60 public class Payment
61 {
62     public virtual void Pay(Money money)
63     {
64         // from base
65     }
66 }
```

```
68 public class CashPayment : Payment
69 {
70     public override void Pay(Money money)
71     {
72         //some code here - pay with cash
73     }
74 }
```

```
76 public class CreditCardPayment : Payment
77 {
78     public override void Pay(Money money)
79     {
80         //some code here - pay with credit card
81     }
82 }
```

Liskov Substitution

Principle

"Let $q(x)$ be a property provable about objects x of type T . Then $q(y)$ should be provable for objects y of type S where S is a subtype of T "

What do you
say?



Liskov Substitution Principle

*“A subclass should **behave** in such a way that it will not cause problems when used instead of the superclass.”*

Liskov Substitution Principle

```
5 public class Employee
6 {
7     public virtual string GetProjectDetails(int employeeId)
8     {
9         Console.WriteLine("base project details");
10    }
11 }
```

```
13 public class CasualEmployee : Employee
14 {
15     public override string GetProjectDetails(int employeeId)
16     {
17         base.GetProjectDetails(employeeId);
18         Console.WriteLine("casual employee project details");
19     }
20 }
```



```
public class ContractualEmployee : Employee
{
    //broken your base class here
    public override string GetProjectDetails(int employeeId)
    {
        Console.WriteLine("contractual employee project details");
    }
}
```

Liskov Substitution Principle

```
5 public class Employee
6 {
7     public virtual string GetProjectDetails(int employeeId)
8     {
9         Console.WriteLine("base project details");
10    }
11 }
```

```
13 public class CasualEmployee : Employee
14 {
15     public override string GetProjectDetails(int employeeId)
16     {
17         base.GetProjectDetails(employeeId);
18         Console.WriteLine("casual employee project details");
19     }
20 }
```

```
public class ContractualEmployee : Employee
{
    public override string GetProjectDetails(int employeeId)
    {
        base.GetProjectDetails(employeeId);
        Console.WriteLine("contractual employee project details");
    }
}
```



Much better

Liskov Substitution Principle

Major examples of LSP violation:

1. Sub-class implements only some methods, other look redundant and ... weird
2. Some methods behavior violates contract
3. equals() method symmetry requirement is violated
4. Subclass throws exception which are not declared by parent class/interface (java prevents from introducing checked exceptions)

Liskov Substitution Principle

```
class Bird extends Animal {  
    @Override  
    public void walk() { ... }  
  
    @Override  
    public void makeOffspring() { ... };  
  
    public void fly() {...} // will look weird for Emu  
}  
  
class Emu extends Bird {  
    public void makeOffspring() {...}  
}
```

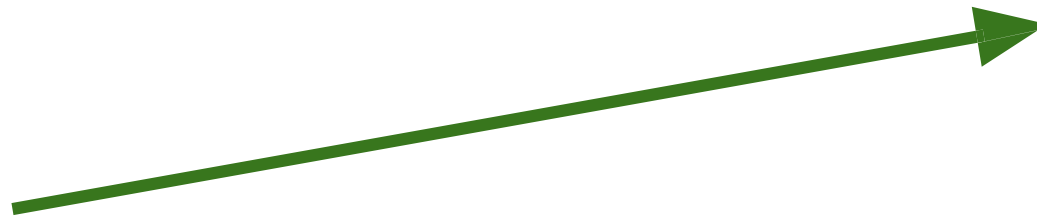


Liskov Substitution Principle

```
class Bird extends Animal {  
    @Override  
    public void walk() { ... }  
  
    @Override  
    public void makeOffspring() { ... };  
}
```

```
class FlyingBird extends Bird {  
    public void fly() {...}  
}
```

```
class Emu extends Bird {  
    public void makeOffspring() {...}  
}
```



Liskov Substitution Principle

```
interface ArraySorter {  
    Object[] sort(Object[] args);  
}  
  
class DefaultArraySorter implements ArraySorter {  
    public Object[] sort(Object[] array) {  
        Object[] result = array.clone();  
        // ...  
    }  
}  
  
class QuickArraySorter implements ArraySorter {  
    public Object[] sort(Object[] array){  
        Object[] result = array;  
        // original array changed! Error! Negative side-effect!  
    }  
}
```

Interface Segregation Principle

*“Clients should **not be forced**
to depend upon interfaces
that they don't use”*

Interface Segregation Principle

```
62 public interface IEmployee
63 {
64     string GetProjectDetails(int employeeId);
65
66     string GetEmployeeDetails(int employeeId);
67 }
68
```

Interface Segregation Principle

```
interface IWorkable
{
    void Work();
    void Eat();
}
```

WHY?????
I don't need you!!

```
class Employee : IWorkable
{
    public void Work()
    {
        Console.WriteLine("Employee is working");
    }

    public void Eat()
    {
        Console.WriteLine("Employee is eating");
    }
}

class Robot : IWorkable
{
    public void Work()
    {
        Console.WriteLine("Robot is working");
    }

    public void Eat()
    {
        Console.WriteLine("Employee is eating");
    }
}
```



Interface Segregation Principle

How to solve this?



Interface Segregation Principle

```
interface IWorkable
{
    void Work();
}

interface IEatable
{
    void Eat();
}
```



You need to create
two interfaces

Interface Segregation Principle

```
interface IWorkable
{
    void Work();
}

interface IEatable
{
    void Eat();
}
```

```
class Employee : IWorkable, IEatable
{
    public void Work()
    {
        Console.WriteLine("Employee is working");
    }

    public void Eat()
    {
        Console.WriteLine("Employee is eating");
    }
}

class Robot : IWorkable
{
    public void Work()
    {
        Console.WriteLine("Robot is working");
    }
}
```

Dependency Inversion Principle

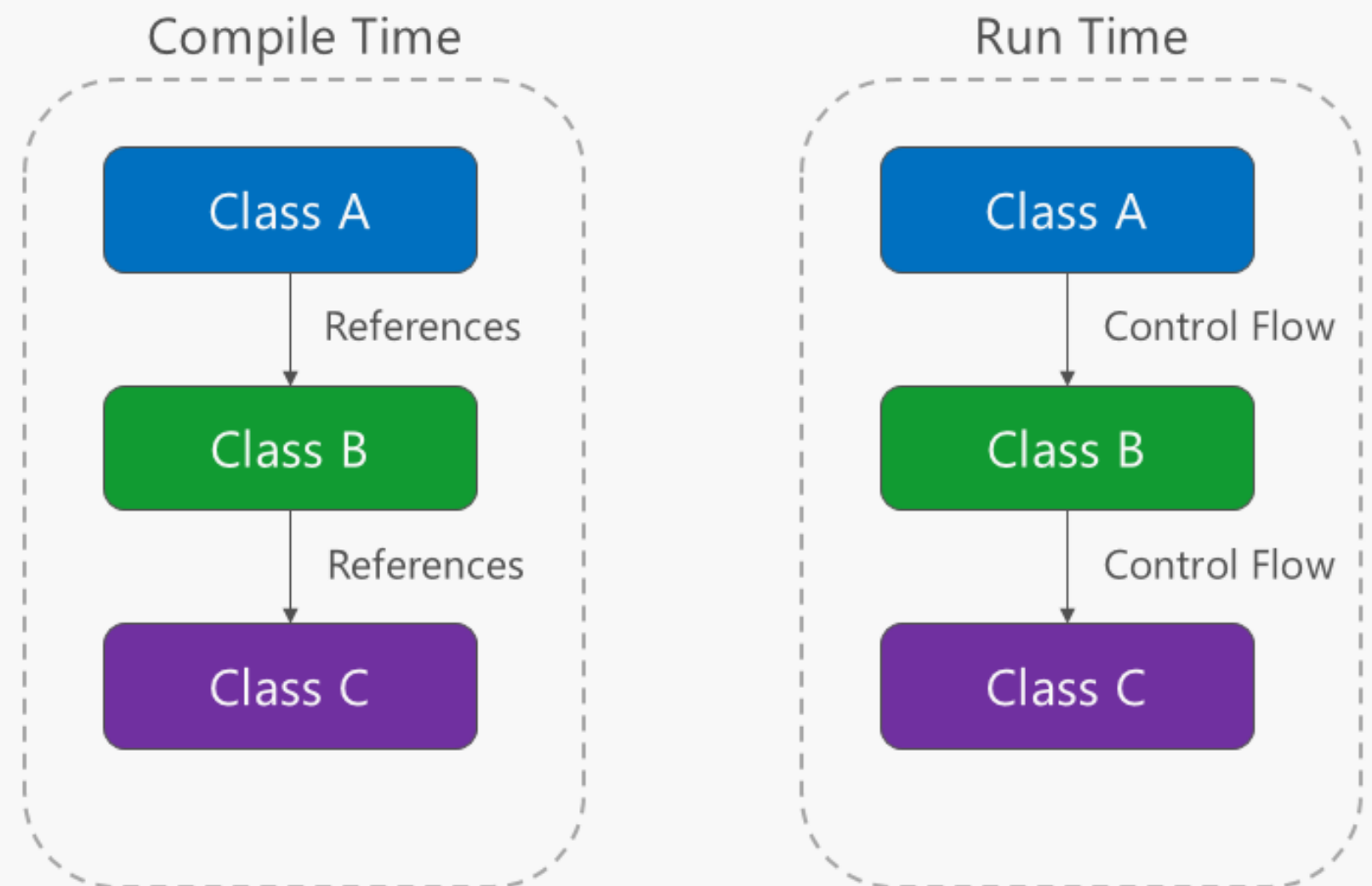
“High-level modules should not depend on low-level modules. Both should depend on abstractions.”

“Abstractions should not depend upon details. Details should depend upon abstractions.”

Direct dependency

If class A calls a method of class B and class B calls a method of class C, then at compile time class A will depend on class B and class B will depend on class C

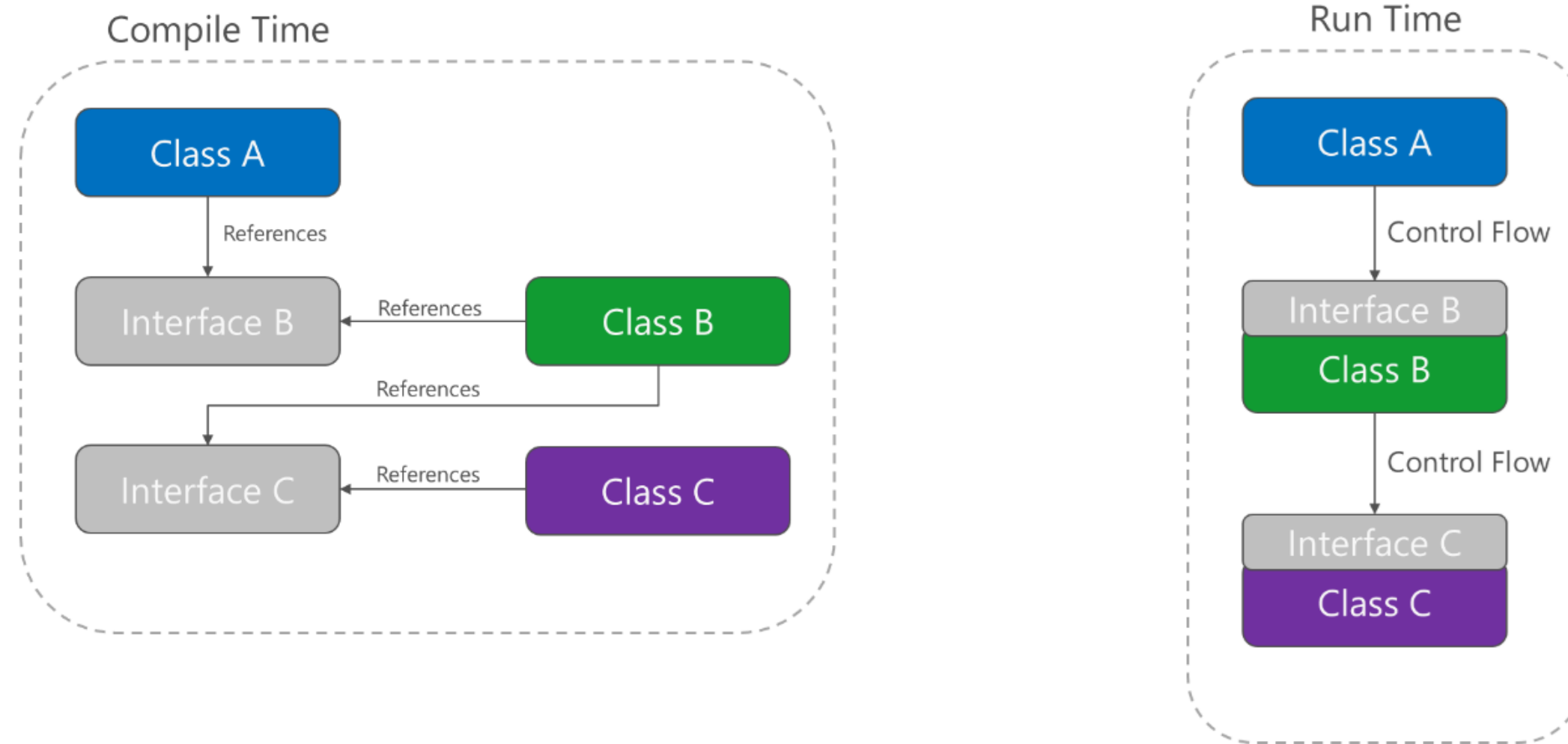
Direct Dependency Graph



Dependency Inversion Principle

Applying the dependency inversion principle allows A to call methods on an abstraction that B implements, making it possible for A to call B at run time, but for B to depend on an interface controlled by A at compile time (thus, inverting the typical compile-time dependency). **Different implementations of these interfaces can easily be plugged in.**

Inverted Dependency Graph



Dependency Inversion Principle

```
175 public class Email
176 {
177     public void SendEmail()
178     {
179         // code to send mail
180     }
181 }
```

```
183 public class Notification
184 {
185     private Email _email;
186     public Notification()
187     {
188         _email = new Email();
189     }
190
191     public void PromotionalNotification()
192     {
193         _email.SendEmail();
194     }
195 }
```

And if I need to send a notification by SMS?
You need to change this.



Dependency Inversion Principle

```
199 public interface IMessenger
200 {
201     void SendMessage();
202 }
```

So, I create an interface and now?

```
204 public class Email : IMessenger
205 {
206     public void SendMessage()
207     {
208         // code to send email
209     }
210 }
```

```
212 public class SMS : IMessenger
213 {
214     public void SendMessage()
215     {
216         // code to send SMS
217     }
218 }
```

Dependency Inversion Principle

```
220 public class Notification
221 {
222     private IMessenger _iMessenger;
223     public Notification()
224     {
225         _iMessenger = new Email();
226     }
227     public void DoNotify()
228     {
229         _iMessenger.SendMessage();
230     }
231 }
```



Dependency Inversion Principle

Constructor injection:

```
235 public class Notification
236 {
237     private IMessenger _iMessenger;
238     public Notification(IMessenger pMessenger)
239     {
240         _iMessenger = pMessenger;
241     }
242     public void DoNotify()
243     {
244         _iMessenger.SendMessage();
245     }
246 }
```

Dependency Inversion Principle

Property injection:

```
248 public class Notification
249 {
250     private IMessenger _iMessenger;
251
252     public IMessenger MessageService
253     {
254         private get;
255         set
256         {
257             _iMessenger = value;
258         }
259     }
260
261     public void DoNotify()
262     {
263         _iMessenger.SendMessage();
264     }
265 }
```

Dependency Inversion Principle

Method injection:

```
268 public class Notification
269 {
270     public void DoNotify(IMessenger pMessenger)
271     {
272         pMessenger.SendMessage();
273     }
274 }
```

Keep in
mind

DRY - Don't repeat yourself

+

SLAP - Single layer abstraction principle

+

SOLID

BEST DEVELOPER



Start your future at EIU

Q&A

Start your future at EIU

THANK YOU

Sample text